

Guión de prácticas

Desarrollo de un sistema RAG para recomendación basada en contenido

Sistemas de Recuperación y Recomendación

Máster en Ciencia de Datos e Ingeniería de Computadores

Objetivos

El objetivo principal de esta práctica es diseñar, implementar y evaluar un Sistema de Recomendación (SR) basado en Contenido (SRC) mediante una arquitectura RAG (Retrieval-Augmented Generation), integrando técnicas de Recuperación de Información (IR), concretamente recuperación semántica vectorial, con modelos de lenguaje de gran escala (LLM) de código abierto para proporcionar recomendaciones personalizadas y explicables.

Este objetivo se divide en los siguientes sub objetivos más específicos:

1. Entender el concepto de RAG.
2. Comprender cómo los SR pueden beneficiarse de la técnica RAG.
3. Conocer los diferentes módulos de un SRC basado en RAG.
4. Ser capaz de desarrollar un SRC basado en RAG.

Introducción

Un sistema de Generación Aumentada por Recuperación (Retrieval-Augmented Generation, RAG) es una arquitectura que optimiza la salida de un Modelo de Lenguaje de Gran Escala (Large Language Model, LLM) consultando una fuente de conocimiento externa y autorizada antes de generar una respuesta. A diferencia de un LLM estándar, que emplea únicamente la información suministrada durante su entrenamiento, un sistema RAG primero busca información relevante en un catálogo (recuperación) y luego usa esa información para responder (generación).

Dicho de otra forma, es un enfoque híbrido que combina un sistema de RI con un LLM. En lugar de generar respuestas basándose únicamente en el conocimiento implícito aprendido durante el entrenamiento, el modelo primero consulta una base de conocimientos externa (por ejemplo, una colección de documentos, descripciones de ítems, perfiles de usuario, etc.) para obtener fragmentos relevantes al contexto de la consulta o petición. Luego, el LLM utiliza esos fragmentos recuperados como contexto adicional para generar una respuesta más precisa, informada y actualizada.

Este proceso suele seguir estos pasos:

1. Entrada: el usuario plantea una consulta o solicitud.
2. Recuperación: un sistema de retrieval (basado en embeddings, BM25, etc.) busca en una base de datos vectorial o corpus los fragmentos más relevantes.
3. Aumento: los fragmentos recuperados se concatenan con la consulta original para formar un prompt enriquecido.
4. Generación: el LLM produce una respuesta basándose en el prompt aumentado.

Algunos usos de esta tecnología son los siguientes:

- Chatbots de atención al cliente: recuperan artículos de la base de conocimiento de la empresa para responder preguntas específicas sobre productos o políticas.
- Asistentes de investigación: consultan artículos académicos o documentación técnica para proporcionar respuestas con referencias actualizadas.
- Generación de informes: combinan datos de fuentes internas (bases de datos, informes previos, etc.) para redactar documentos actualizados.
- Recomendación personalizada de ítems: el sistema recupera descripciones de ítems (películas, productos, artículos, etc.) que coinciden con los intereses explícitos o implícitos del usuario (historial de interacciones, perfil, etc.). Luego, un LLM genera una explicación personalizada justificando el motivo por el que recomienda cada ítem, o incluso genera una lista ordenada con justificaciones.
- Generación de descripciones enriquecidas: para ítems poco conocidos, se pueden recuperar reseñas o datos complementarios y generar una descripción atractiva y contextualizada para el usuario.
- Filtrado colaborativo aumentado: se pueden recuperar opiniones de usuarios similares al actual y el LLM las sintetiza para recomendar ítems que puedan gustarle.

Algunas ventajas de RAG frente al uso de un LLM normal son las siguientes:

1. Acceso a información actualizada y específica. Los LLM tradicionales están limitados a su fecha de entrenamiento y no conocen ítems nuevos ni cambios en catálogos. RAG permite consultar una base de datos actualizada en tiempo real, asegurando que las recomendaciones incluyan los últimos lanzamientos o modificaciones.

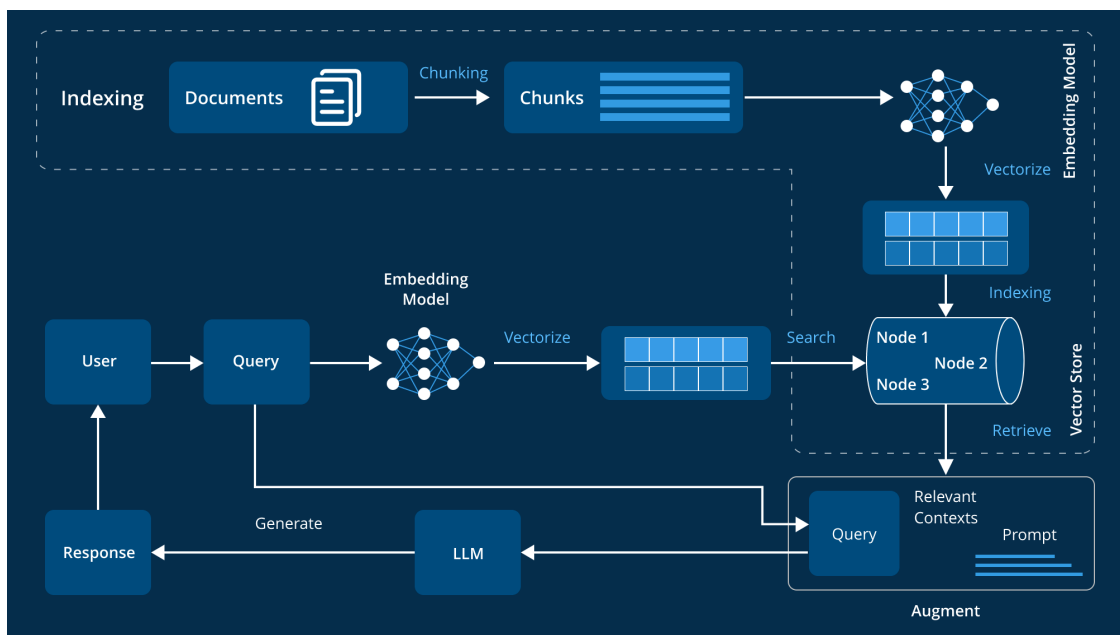
2. Reducción de alucinaciones. Al proporcionar al LLM fragmentos reales de la base de conocimiento, se reduce la probabilidad de que invente ítems o atributos inexistentes, algo crítico en recomendaciones donde la precisión es esencial.
3. Personalización basada en datos del usuario. Un LLM normal no puede acceder directamente al historial de un usuario sin ser afinado. Con RAG, se pueden recuperar interacciones pasadas, preferencias explícitas o datos demográficos del usuario y usarlos como contexto, generando recomendaciones verdaderamente personalizadas.
4. Explicabilidad y trazabilidad. Las respuestas generadas pueden citar las fuentes recuperadas (descripciones, reseñas, etc.), lo que permite al usuario entender los motivos por los que se le recomienda algo. Esto aumenta la confianza y facilita la depuración del sistema.
5. Manejo eficiente de catálogos grandes y de larga cola. Los LLM tienen una capacidad limitada de contexto y no pueden "conocer" todos los ítems de un catálogo extenso. RAG permite recuperar solo los ítems relevantes para cada consulta, haciendo factible trabajar con millones de productos.
6. Flexibilidad para dominios específicos. Se puede construir una base de conocimiento con información de dominio (por ejemplo, características técnicas de productos, reseñas de expertos, etc.) sin necesidad de reentrenar el LLM. Esto facilita la adaptación a distintos sectores (cine, libros, e-commerce, etc.).
7. Menor costo computacional y de mantenimiento. Reentrenar un LLM para cada dominio o actualización es costoso. Con RAG, solo es necesario actualizar la base de conocimiento, mientras que el LLM puede permanecer genérico.
8. Mejora en la diversidad y serendipia. Al recuperar información variada de la base de conocimiento, el sistema puede sugerir ítems menos obvios pero relevantes, enriqueciendo la experiencia del usuario.

Un sistema de este tipo se puede estructurar en los siguientes módulos principales:

- **Módulo I: procesamiento de ítems.** Es el encargado de limpiar el contenido textual de los ítems, concatenar metadatos relevantes y convertir los textos en vectores semánticos densos.
- **Módulo II: indexación y recuperación.** Los vectores deberán ser almacenados e indexados en una base de datos vectorial con el objetivo de poder acceder a ellos de manera eficiente. Llegada una consulta (en nuestro caso un perfil de usuario), el motor de recuperación se encargará de convertirla en vector y obtener una lista de ítems ordenados de forma decreciente a la relevancia con respecto al perfil del usuario.

- **Módulo III: aumentación.** A partir del texto contenido en el perfil de usuario y de los top-k ítems del ranking, se genera un prompt que empaquete los datos con la petición al LLM.
- **Módulo IV: generación.** El LLM recibe el prompt (indicaciones + datos de los ítems) y genera una nueva ordenación de los ítems introducidos y la justificación de su idoneidad.
- **Módulo V: filtro de alucinaciones.** Comprueba que los nombres generados por el LLM existan realmente en la entrada.
- **Módulo VI: evaluación.** En un entorno de pruebas se calculan las métricas de rendimiento de la recuperación.

Gráficamente, la arquitectura se podría resumir en el siguiente gráfico ([Fuente](#)):



Sesiones de la práctica

En esta práctica partiremos de un conjunto de datos relacionado con la recomendación, compuesto por productos de cualquier tipo y una descripción textual de los mismos. También será necesario contar con usuario y sus perfiles que serán la unión, en un único texto, de las descripciones textuales de los productos consumidos (visitados, accedidos, comprado, etc.) por dicho usuario. De forma genérica en la terminología RAG, los perfiles de usuario harán las veces de consultas al SRI.

A partir de ahí se desarrollará un sistema RAG completo. Toda la práctica se podrá hacer en Python y se llevará a cabo en tres sesiones presenciales.

Sesión 1: procesamiento de ítems, generación de embeddings e indexación.

En primer lugar, hay que seleccionar una colección que sea útil para un problema de recomendación de contenido. Algunas colecciones que pueden ser interesantes las podéis encontrar [aquí](#), además de la famosa [Movielens](#). A la hora

de elegir, tened en cuenta que el *dataset* tenga texto y que no esté orientado a la evaluación del filtrado colaborativo.

El preprocesado consistirá en la eliminación de etiquetas, si la colección viene en HTML, y el paso a minúscula. Seguidamente se concatenan los metadatos y contenido para formar un “documento” por ítem. En el caso de que la longitud de los textos sea muy larga (por ejemplo, libros) se tendría que aplicar alguna técnica de troceado (chunking) para adaptarnos al contexto del LLM.

Para la generación de vectores se elegirá cualquier modelo que se vea conveniente, aunque se recomienda alguno ligero como *all-MiniLM-L6-v2*, incluido en la biblioteca *sentence-transformer* de Python. Este genera vectores con 384 dimensiones en la CPU de vuestro ordenador.

Con objeto de realizar una búsqueda semántica eficiente, es necesario almacenar los vectores en algún tipo de base de datos vectorial. En este caso se propone *FAISS* (*Facebook AI Similarity Search*), la cual genera estructuras de datos espaciales que permiten realizar búsquedas en tiempo logarítmico en lugar de lineal. Para indexar los vectores se propone el índice *IndexFlatIP*. Para posteriormente, poder emplear la similitud del coseno en la fase de búsqueda, los vectores deben normalizarse a 1. Así el producto escalar que ofrece FAISS (Inner Product, IP) es equivalente a la medida del coseno del ángulo entre los vectores. La normalización se puede hacer mediante *faiss.normalize_L2(vectores)* y la creación del índice *faiss.IndexFlatIP(d)*.

Sesión 2: recuperación

Dada la base de datos ya generada de vectores con todos los ítems de la colección, la siguiente fase es la de recuperación de ítems dada una consulta. En nuestro caso, este será un perfil de usuario, es decir, los productos que ha visto, comprado, consumido previamente en su interacción con el sistema.

Estos perfiles no suelen venir habitualmente en los *datasets* por lo que hay que construirlos y su construcción dependerá de la información que tengamos disponible en el conjunto de datos que se ha elegido. Podría ser un usuario que aparezca en él. Por ejemplo, en MovieLens, se seleccionaría un usuario con varias películas valoradas y crearemos su perfil concatenando la descripción textual de esas películas. Otra opción sería crear perfiles de usuario de personas sintéticas usando prompts a LLMs, solicitando perfiles contradictorios: “Busco películas de animación clásica, preferiblemente de los años 80 o 90, que tengan una temática de aventura y amistad, evitando el estilo 3D moderno.”, “Me interesa el cine de ciencia ficción hard, donde la física sea realista y el conflicto sea más intelectual que de acción.” ó “Mi perfil de preferencia es [GÉNERO], pero con un tono [TONO]. Me interesan especialmente los temas relacionados con [TEMÁTICA].”. Adapta estos prompts a tu conjunto de datos y genera 5 o 6 perfiles de usuarios para probar.

Una vez generados los perfiles, se deben convertir a vectores y, usando FAISS, proceder a la recuperación. Este permite generar una matriz de consultas y

pasar al recuperador dicha matriz en vez de hacerlo de forma iterativa (*index.search(queries, k)*). FAISS devolverá una matriz con las puntuaciones de similitud de cada usuario con los ítems y, a la vez, los IDs recomendados para cada usuario.

Sesión 3: aumentación, generación y evaluación

De la etapa anterior se genera un ranking de productos para cada usuario. Este ranking (*top-k* ítems) será la base para generar un *prompt* al LLM para que haga la recomendación final y la justificación.

En el prompt al LLM se tendrá que indicar algo así como sigue:

- Rol: “Actúas como un sistema de recomendación de [TEMÁTICA]. Tu objetivo es analizar el perfil de un usuario y seleccionar, de entre una lista de candidatos recomendados, las opciones que mejor encajen con sus gustos actuales.
- Contexto del usuario: el perfil de usuario. “El usuario ha definido sus preferencias actuales como: “{perfil_usuario}””.
- Candidatos recuperados: “A continuación se presentan los ítems más similares encontrados en nuestro catálogo: “{lista_items_recuperados}””.
- Instrucciones de generación:
 1. RE-RANKING. Ordena los 3 mejores ítems de mayor a menor afinidad real con el contexto del usuario.
 2. JUSTIFICACIÓN. Para cada ítem, explica en una frase corta el motivo por el que encaja con lo que el usuario busca, mencionando detalles específicos de la sinopsis o metadatos.
 3. FILTRO DE CALIDAD. Si un ítem recuperado es totalmente irrelevante, ignóralo.
 4. INTEGRIDAD. No inventes películas que no estén en la lista de candidatos.
- Formato de salida:

Devuelve la respuesta estrictamente en este formato JSON:

```
[  
  {  
    "ranking": 1,  
    "titulo": "Nombre del ítem",  
    "razonamiento": "Explicación breve..."  
  },  
  ...  
]
```

Para esta última parte de GenAI, se puede emplear la biblioteca *ollama* de Python y algún modelo básico como *llama3.2*, *phi3* o *mistral*. Alternativamente, se puede emplear alguna API como la de *Hugging Face Inference API* (biblioteca *huggingface_hub*) o la de *Google Gemini API* para acceder a *Gemini 1.5 Flash*.

Por último, una vez obtenida la salida por parte de la GenAI, en forma de JSON, se deberá comprobar que el LLM no ha alucinado y que todos los ítems devueltos estaban en la lista de entrada. Además es interesante comparar los rankings de ítems, el de entrada y el de salida, e informar sobre cuántas veces el LLM cambió el orden propuesto por FAISS.

Entrega

Se entregará, en la tarea correspondiente de Prado, el código fuente y un informe lo suficientemente completo para que se describa la práctica realizada detalladamente y las decisiones tomadas y justificadas (sin excederse de 10 páginas), además de un párrafo que evalúe la calidad de la salida del RAG. No se debe meter código fuente en dicho informe salvo algunas excepciones donde sea interesante para explicar algo relevante.

Bibliografía

Algunas fuentes interesantes para conocer más sobre esta técnica son las siguientes:

- Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS 2020). <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- Zhao, P., et al. (2025). Retrieval-Augmented Generation for AI-Generated Content: A Survey. Data Science and Engineering. <https://link.springer.com/article/10.1007/s41019-025-00335-5>
- Hugging Face. (2020). RAG Documentation. Transformers Library. https://huggingface.co/docs/transformers/en/model_doc/rag
- Pinecone - Retrieval-Augmented Generation (RAG) Guide <https://www.pinecone.io/learn/retrieval-augmented-generation/>
- IBM Research - What is retrieval-augmented generation? <https://research.ibm.com/blog/retrieval-augmented-generation-rag>
- LangChain Blog - RAG Strategies <https://blog.langchain.dev> (buscar por RAG)
- Amazon Web Services (AWS). (2025). ¿Qué es la generación aumentada por recuperación (RAG)? <https://aws.amazon.com/es/what-is/retrieval-augmented-generation/>

- Google Cloud. (2025). ¿Qué es la generación aumentada de recuperación (RAG)?
<https://cloud.google.com/use-cases/retrieval-augmented-generation?hl=es>
- Romero, A. (2025). Aprovechando al máximo nuestros datos con los grandes modelos de lenguaje: Introducción a RAG.
<https://medium.com/@augustoromero/aprovechando-al-m%C3%A1ximo-nuestros-datos-con-los-grandes-modelos-de-lenguaje-a467a60d651e>